

JK AI Student Lab Assignment

Batch Hardware Integrity & Counterfeit Audit System

COURSE MODULE

Expert Systems & Security

INSTRUCTOR

Laurie MAVOUNGOU (CEO)

TARGET AUDIENCE

AI Engineering Students

Lab Overview

In this lab, you will expand the core **Hardware Integrity Expert System** from auditing a single local computer (via Windows Management Instrumentation) to running **batch supply chain audits** across hundreds of devices simultaneously. By utilizing structured component and device datasets from **Kaggle**, you will simulate enterprise-scale counterfeit detection, verifying whether hardware manifests match expected factory baselines.

Learning Objectives

- **Scale Inference Engines:** Adapt a rule-based expert system (`working_memory()`) to process iterable batches of device data rather than a single live host.
- **Data Engineering & Normalization (ETL):** Ingest raw tabular or JSON data from public repositories and transform them into the standard dictionary schema required by the inference engine.
- **Supply Chain Security:** Apply AI reasoning to catch forged part numbers, mismatched RAM capacities, and cloned serial numbers designed to mimic original equipment manufacturer (OEM) standards.

Assignment Instructions & Tasks

Task 1: Choose Your Dataset (Kaggle Integration)

To complete this lab, you will source hardware data from Kaggle. You may choose from recommended datasets or **bring your own custom Kaggle dataset** related to computer hardware, PC parts, or laptop specifications.

- **Recommended Starting Datasets:**
 - [PC Parts Dataset](#) (JSON/structured files split by component types like motherboards, CPUs, and memory)
 - [Laptop Prices and Specifications Dataset](#) (Tabular CSV files detailing processors, RAM configurations, and storage types)
- **Custom Dataset Bonus:** You are strongly encouraged to search Kaggle for other hardware inventory, server manifest, or component specification datasets to test your system against diverse hardware profiles.

Task 2: Data Ingestion & Schema Mapping

Write a Python script using pandas (or native JSON/CSV parsers) to load your chosen Kaggle dataset. Map the rows of the dataset into the standard dictionary structure used by the expert system's fact acquisition layer:

```
device_manifest = { "bios": {"manufacturer": "...", "serial_number": "...", "version": "..."}, "motherboard": {"manufacturer": "...", "product": "...", "serial_number": "..."}, "processors": [{"manufacturer": "...", "name": "...", "processor_id": "...}], "memory": [{"manufacturer": "...", "part_number": "...", "serial_number": "...", "capacity": ...}], "disks": [{"model": "...", "manufacturer": "...", "serial_number": "...", "interface": "..."}] }
```

Task 3: Implement Batch Inference

Modify the expert system's main loop so that instead of querying a single local machine via `computer = WMI()`, it loops through every device record in your Kaggle dataset.

- Compare each device record against a centralized expected baseline database (or a baseline dictionary representing factory standards).
- Collect all flagged inconsistencies using the `working_memory()` function.

Task 4: Generate the Audit Report

Output a comprehensive batch audit summary report (in CSV or JSON format) that categorizes each device tested as either:

- **CONSISTENT**: Passes all validation rules.
- **COMPROMISED / COUNTERFEIT**: Fails one or more verification rules (with exact reasons listed, such as *Memory partNumber does not match* or *Disk serialNumber does not match*).

Bonus Points Criteria (Extra Credit)

Students can earn up to **20 bonus points** by implementing any of the following enhancements:

- **Multi-Dataset Integration**: Merge two different Kaggle datasets (e.g., combining a CPU benchmark dataset with a motherboard dataset) to create complex, cross-referenced hardware audits.
- **Automated Scoring Dashboard**: Write a script that calculates a "Counterfeit Risk Score" percentage for each device batch based on the severity of the flagged anomalies.

Ready to Begin?

Download your chosen dataset from Kaggle, configure your database baseline, and run your batch inference pipeline!